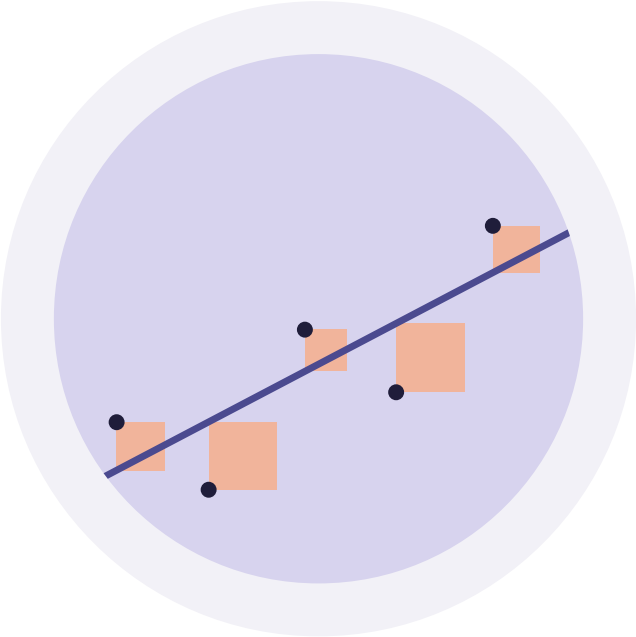
Линейная регрессия

Урок



Третий из пяти уроков о моделях. Из урока «Ближайшие соседи» здесь нужны train и test, среднеквадратичная ошибка MSE и стандартизация признаков. Пример — те же двенадцать пингвинов, но ответ теперь число: масса в граммах по длине крыла в миллиметрах, а в конце — по острову, где пингвин живёт.

```
import numpy as np
# крыло, мм
fl = np.array([185, 190, 182, 195, 193, 198,
               189, 201, 214, 221, 210, 224])
# масса, г
m = np.array([3600, 3800, 3300, 3950, 3550, 3800,
              3400, 3950, 4900, 5400, 4650, 5600])
```

1 Линейная регрессия

Линейная регрессия — $a(x) = w_1 x_1 + \dots + w_d x_d + b = w \cdot x + b$; веса w и *свободный член* b подбирают так, чтобы MSE на train была наименьшей: это *метод наименьших квадратов*. Свободный член прячут в веса: каждому объекту приписывают признак, всегда равный 1, и b становится его весом. Дальше X — таблица $n \times (d+1)$ со столбцом единиц, $w \in \mathbb{R}^{d+1}$, ответы модели на всех объектах — вектор Xw, и

$$L(w) = \frac{1}{n}\sum_{i=1}^{n}(x_i \cdot w - y_i)^2 = \frac{1}{n}\|Xw - y\|^2.$$

Утверждение. Минимум L — на решениях *нормальных уравнений* $X^\top X w = X^\top y$. Если столбцы X линейно независимы, решение одно: $w = (X^\top X)^{-1} X^\top y$.

Почему так. Производная по весу w_j — $\frac{\partial L}{\partial w_j} = \frac{2}{n}\sum_i (x_i \cdot w - y_i)\, x_{ij}$, и все производные вместе — *градиент* $\nabla L = \frac{2}{n} X^\top (Xw - y)$: координата j вектора $X^\top r$ — сумма произведений столбца j на r. В минимуме градиент

нулевой, отсюда уравнения. Обратно, пусть $X^\top X w^* = X^\top y$; для любого v

$$\|X(w^* + v) - y\|^2 = \|Xw^* - y\|^2 + 2v^\top X^\top (Xw^* - y) + \|Xv\|^2,$$

среднее слагаемое равно нулю, последнее неотрицательно, и $L(w^* + v) \geq L(w^*)$ — с равенством только при $Xv = 0$. При независимых столбцах это лишь $v = 0$: минимум один. И $X^\top X$ обратима: из $X^\top X v = 0$ следовало бы $\|Xv\|^2 = v^\top X^\top X v = 0$.

При одном признаке уравнения решаются руками. Первое, по столбцу единиц, даёт $b = \bar{y} - w\bar{x}$ — прямая проходит через точку средних; подстановка во второе — $w = \sum_i (x_i - \bar{x})(y_i - \bar{y}) / \sum_i (x_i - \bar{x})^2$. Масса по длине крыла — кодом, и каждый миллиметр крыла выходит в 54 грамма массы; обратную матрицу не считают, `solve` решает систему точнее и быстрее:

```
A = np.column_stack([np.ones(12), fl])
w = np.linalg.solve(A.T @ A, A.T @ m)
print(w.round(2), ((A @ w - m) ** 2).mean().round(1))
```

```
[-6725.05    54.37] 29890.7
```

Гребневая регрессия

Если признаки почти зависимы — скажем, крыло в миллиметрах и оно же в сантиметрах с округлением, — $X^\top X$ почти вырождена: веса огромны, разных знаков и сильно меняются от малой правки данных. *Гребневая регрессия*, ridge, штрафует большие веса:

$$L_\lambda(w) = \|Xw - y\|^2 + \lambda \|w\|^2, \qquad (X^\top X + \lambda E)\, w = X^\top y.$$

Уравнение выводится так же, градиент штрафа — $2\lambda w$. При $\lambda > 0$ матрица обратима всегда: $v^\top (X^\top X + \lambda E)\, v = \|Xv\|^2 + \lambda \|v\|^2 > 0$ при $v \neq 0$. Ошибка здесь — сумма квадратов, как в sklearn; к среднему переходят делением λ на n.

В формуле штрафуются все веса, но на деле свободный член не штрафуют. Стянутый к нулю b тянет к нулю все ответы, и результат зависит от того, откуда отсчитывать y: от нуля граммов или от трёх килограммов. От сдвига y не зависят только наклоны, и штраф осмыслен на них. Сами наклоны зависят от единиц признаков, поэтому перед ridge признаки стандартизуют.

Градиентный спуск

При тысячах признаков систему решать дорого, а у большинства моделей готовой формулы нет вовсе. Общий способ — *градиентный спуск*, шаг против градиента: $w \leftarrow w - \eta \nabla L(w)$, где $\eta > 0$ — *шаг обучения*. Градиент показывает, куда L растёт быстрее всего, и малый шаг против него уменьшает L. Спуск для массы по крылу, признак стандартизован:

```
s = (fl - fl.mean()) / fl.std()
A = np.column_stack([np.ones(12), s])
w = np.zeros(2)
for step in range(100):
    w = w - 0.1 * 2 / 12 * A.T @ (A @ w - m)
print(w.round(2))
print(np.linalg.solve(A.T @ A, A.T @ m).round(2))
```

```
[4158.33  729.75]
[4158.33  729.75]
```

Спуск пришёл туда же, куда `solve`, и стандартизация здесь не для красоты. У стандартного признака среднее 0 и средний квадрат 1, поэтому $\frac{1}{n}A^\top A = E$, градиент — $2(w - w^*)$, и шаг умножает расстояние до ответа w^* на $1 - 2\eta = 0{,}8$: за сто шагов — в $0{,}8^{100} \approx 2 \cdot 10^{-10}$ раз. При $\eta > 1$ множитель по модулю больше 1, и спуск разлетается. В миллиметрах собственные числа $\frac{1}{n}A^\top A$ — около $4 \cdot 10^4$ и

$4 \cdot 10^{-3}$: шаг, не разлетающийся вдоль первого направления, вдоль второго почти стоит.

Категория: номер или индикаторы

Признак-категория — остров, где живёт пингвин: Бискоу, Дрим или Торгерсен. Номера 0, 1, 2 говорят линейной модели, что острова стоят на прямой в этом порядке и масса от острова к острову меняется на одно и то же, — хотя порядок номеров случаен. Правильно — *индикаторы*, one-hot: по столбцу на значение, 1 в строках этого острова и 0 в остальных. Тогда у каждого острова свой вес.

```
isl = np.array([2, 1, 0, 2, 1, 1, 1, 1, 0, 0, 0, 0])
A = np.column_stack([np.ones(12), isl])
w = np.linalg.solve(A.T @ A, A.T @ m)
print((w[0] + w[1] * np.arange(3)).round(1))
H = np.eye(3)[isl]
print(np.linalg.solve(H.T @ H, H.T @ m))
```

```
[4617.3 4005.3 3393.3]
[4770. 3700. 3775.]
```

`np.eye(3)[isl]` — строки единичной матрицы по номерам, готовые индикаторы. С ними ответ на острове — средняя масса его пингвинов, а с номерами ответы стоят на одной прямой, и Торгерсену досталось 3393 вместо 3775. Столбца единиц рядом с индикаторами нет: их сумма в каждой строке и так 1, и вместе со свободным членом столбцы зависимы. Поэтому либо свободного члена нет, либо один индикатор выбрасывают.

2 То же в sklearn

Порядок вызовов — как в уроке «Ближайшие соседи»: создать объект с гиперпараметрами, `fit(X, y)`, `predict(X)`. `LinearRegression` — тот же метод наименьших квадратов,

свободный член — `intercept_`, веса — `coef_`. `Ridge(alpha=...)` штрафует $\alpha\|w\|^2$ при сумме квадратов ошибок и свободный член не трогает:

```
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Ridge

lr = LinearRegression().fit(fl[:, None], m)
print(lr.coef_.round(2), lr.intercept_.round(2))
for a in [1, 100, 10000]:
    r = Ridge(alpha=a).fit(s[:, None], m)
    print(a, r.coef_.round(1), r.intercept_.round(1))
```

```
[54.37] -6725.05
1 [673.6] 4158.3
100 [78.2] 4158.3
10000 [0.9] 4158.3
```

Наклон уходит к нулю, а свободный член остаётся средней массой: модель сжимается к константе, а не к нулю. Этим `Ridge` и расходится с формулой раздела 1, где штраф стоит на всех весах: в задаче, где правило задано формулой, считают по ней. Один признак sklearn ждёт столбцом, `fl[:, None]`, а не рядом.

Дальше — урок «Логистическая регрессия и перцептрон»: та же линейная формула, но ответ — класс.